음성 명령과 음원 방향 정보를 활용한 실내 가정용 로봇 이동 제어 시스템

Voice-Commanded Indoor Robot Control Using Sound Source Direction

지능형로보틱스 (39159-01) 최종 프로젝트
배지혜 (2491016)

동기 및 목표

동기

- 사람은 서로 음성으로 소통하며 , 소리가 들려오면 그 방향으로 고개를 돌려 확인하는 등 소리의 방향 정보를 활용해 반응한다 .
- 인간과 로봇의 자연스러운 상호작용을 위해서는 로봇이 음성의 의미와 함께 그 방향 정보에 따라 행동하도록 만드는 것이 필요하다 .
- 이는 인간 중심적인 환경에서 작동하는 실내 가정용 어시스턴트 로봇에게 더욱 중요한 요소가 된다 .

동기 및 목표

목표

- 본 프로젝트는 로봇의 동작에 소리와 공간적 맥락을 반영시킴으로써 , 가정 환경에서 별도의 입력 장치 없이도 음성을 통해 사용자와 로봇간의 직관적인 상호작용이 가능한 가정용 어시스턴트 로봇 시스템을 시뮬레이션 한다 .
- 음성 명령만을 움직임에 대응시키는 기존 프로젝트 * 를 확장하여 , 본 프로젝트는 추가적으로 음원의 방향을 계산하여 로봇의 행동에 매핑한다 .

* https://github.com/arynfs/Voice-Controlled-Bot-Using-ROS2

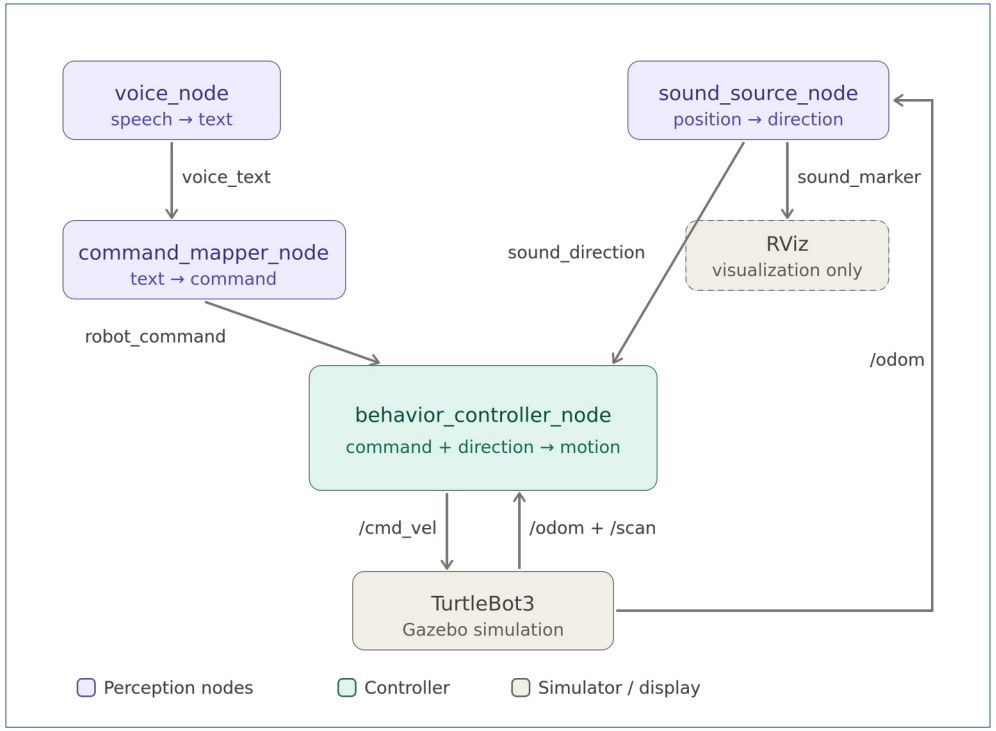
최종 목표

- (1) 사용자가 입력한 음성에서 명령어를 인식 ,
- (2) 사용자가 입력한 음원의 방향을 계산 ,
- (3) 피드백 루프를 통해 목표 방향까지 안정적으로 회전 ,
- (4) 네 가지 명령이 소리의 방향을 기준으로 실행되도록 동작 구현

시스템 구조

노드

- voice_node
- command_node
- sound_source_node
- behavior_controller_node



사이를 연결하는 토픽

- voice_text
- robot_command
- sound_direction
- /odom
- /scan
- /cmd_vel
- sound_marker

시스템 구조 — 토픽

토픽	발행자	구독자
voice_text	voice_node	command_mapper_node
robot_command	command_mapper_node	behavior_controller_node
sound_direction	sound_source_node	behavior_controller_node
sound_marker	sound_source_node	RViz (시각화)
/odom	TurtleBot3 (Gazebo)	sound_source_node, behavior_controller_node
/scan	TurtleBot3 (Gazebo)	behavior_controller_node
/cmd_vel	behavior_controller_node	TurtleBot3 (Gazebo)

시스템 구조 — 노드 역할 / 데이터 흐름

각 노드의 역할

- voice_node: 음성을 인식하고 인식된 텍스트를 voice_text 에 발행
- command_mapper_node: 그 텍스트를 읽어 네 가지 명령과 대조한 뒤 , 명령어 문자열을 robot_command 에 발행
- sound_source_node: 음원 위치를 입력받아 , 로봇의 /odom 에서 얻은 현재 자세를 이용해 그 소리를 향한 방위를 계산하고 이를 sound_direction 에 발행
- behavior_controller_node: 최신 명령과 최신 방향을 로봇의 자세와 결합하여 /cmd_vel 에 속도 명령을 생성한다 .

전체 데이터 흐름

- 음성 → 텍스트 → 명령
- 좌표 → 방향 계산
- 이 두 흐름이 최종적으로 controller 에 입력되는 구조

시스템 구조 — 각 명령어에 대한 동작 정의

- come here: 로봇이 음원 방향을 향하도록 회전한 뒤 앞으로 전진한다 .
- back up: 로봇이 음원의 반대 방향을 향하도록 회전한 뒤 앞으로 전진하여 , 결과적으로 음원에서 멀어지게 한다 .
- turn to me: 로봇이 음원을 향하도록 회전한다 . 이때 로봇이 이미 움직이고 있었다면 회전하면서 계속 전진하고 , 정지 상태였다면 제자리에서 회전한다 .
- stop: 다른 모든 동작을 덮어쓰고 즉시 로봇을 멈춘다 .
- 이들 중 어떤 경우든 전진 동작은 추가로 장애물 안전 검사를 거치며 , 이를 통해 로봇이 가까운 물체에 충돌하지 않도록 한다 .

구현 — voice_node

- 마이크 입력 → 텍스트
- PC 마이크에서 오디오 입력 , 들어오는 샘플을 음성 모델에 전달한다 .
- 음성 모델은 Vosk 모델 중 vosk-model-small-en-us-0.15 모델을 사용하여 실시간으로 음성을 인식 .
- 인식기가 완료된 발화에 대해 비어 있지 않은 텍스트를 반환하면 , 이 노드는 그 텍스트를 voice_text 토픽에 게시

Model	Size	Word error rate / Speed	Notes	License
`vosk-model-small-en-us-0.15`	40M	9.85 (librispeech test-clean) 10.38 (tedlium)	Lightweight wideband model for Android and RPi	Apache 2.0

https://alphacephei.com/vosk/models

구현 — command_node

- voice_text 를 구독 → 자유 형식으로 인식된 텍스트를 네 가지 정규 명령 중 하나로 매핑한다 .
- Python re 모듈을 사용 , 정규표현식 기반 명령 매핑
  - come here: "come" (O) / "welcome" (X)
  - back up: "back up" (O) / "backup" (O)
  - turn to me: "turn…" (E.g., turn, turn to, turned to me) (O)
- 어떤 패턴이 매칭되면 이 노드는 정규 명령 문자열을 robot_command 에 발행하고 , 아무 것도 일치하지 않으면 아무것도 발행하지 않는다 .

구현 — sound_source_node

- 음원 좌표 → absolute direction 계산
- Gazebo 시뮬레이션의 소리 재생 한계 , 대신 사용자가 입력한 좌표를 음원 위치로 사용 .
- 입력된 위치를 기억 , 10 Hz 타이머를 통해 로봇의 현재 자세로부터의 절대 방위를 계속해서 다시 계산해 sound_direction 에 게시 (문제 해결 파트에서 상술)
- 입력된 위치를 RViz 마커로 게시해 시각화
- 발행한 절대 방위와 그에 대응하는 상대 DOA 각도 (사람이 지각하는 방향) 둘 다를 로그로 남긴다 .

구현 — behavior_controller_node (1)

- 인식된 명령어와 음원의 방향 → 동작을 수행
- 가장 최근 명령과 , 목표 방향을 보관 (command latching, target latching)
- /odom 의 quaternion → yaw 변환 , 현재 방향 추적
- 10 Hz 타이머를 사용해 제어 루프를 실행
- 비례 제어 알고리즘
  - 매 틱마다 목표 헤딩과 현재 yaw 의 정규화된 차이를 계산하여 heading error 를 구하고 , (error=target-current_yaw)
  - 각속도를 클램프된 비례항으로 설정한다 . (angular.z = clamp(Kp · error, max))
  - 로봇은 목표 헤딩에 부드럽게 수렴한다 .
- 회전이 허용 오차 안으로 정렬 (alignment) 되었을 때 전진 허용
- 목표 각도는 새 명령이 들어올 때만 갱신

구현 — behavior_controller_node (2)

- 전진 동작 시 장애물 검사 , obstacle stop
  - /scan 을 구독하고 전방 ±30° 의 호 안에서 유효한 최소 거리를 계산한다 .
  - 루프가 전진을 명령하려는 순간 , 먼저 그 호 안에 0.3 m 보다 가까운 물체가 없는지 확인한다 .
  - 장애물이 있으면 선속도만 0 으로 설정 , 전진은 멈추지만 회전은 계속할 수 있도록 구현 .
- 주요 튜닝 파라미터 :

proportional gain of 1.5, a maximum angular velocity of 1.0 rad/s, a forward speed of 0.15 m/s, an alignment tolerance of 0.05 rad (about 2.86°), an obstacle threshold of 0.3 m, and a frontal arc of ±30°, with the control loop and direction republishing both at 10 Hz.

트러블슈팅 및 설계 결정

1. 절대 각도 vs 상대 각도

- 문제 : 상대 각도는 로봇이 회전하는 순간 오래된 값이 됨
- 해결 : 모든 상대 각도는 고정된 프레임에 고정된 절대 헤딩으로 변환 .
  - sound_source_node 수정 : odom 좌표계 기준 절대 방향 publish, 동시에 상대 각도는 로그로 표시

2. Bang-Bang 제어 vs 비례 제어

- 초기에는 TurtleBot3 의 relative_move 예제를 모델로 삼아 , 일정한 각속도로 회전하다가 작은 각도 임계값 안에 들어오면 정해진 순서대로 단계적으로 진행하는 구조 (회전 , 정지 , 직진) 를 고려했다 .
- 문제 : 이동 중 회전 불가 , 즉시 정지 어려움 , 목표 변경 대응 어려움
- 해결 : 실시간으로 오차를 계산하는 비례 제어 사용
- 장점 : 더 부드러운 회전 , 즉각적인 반응

트러블슈팅 — 3. 음성 명령 인식 오류 보완

- 테스트 중 두 가지 인식 문제
- 단순 부분 문자열 매칭은 오류를 일으킴
  - 예를 들어 인식된 텍스트 안에 문자열 "come" 이 포함되어 있는지만 검사하면 , "welcome" 같은 무관한 단어가 "come here" 명령을 유발
  - 해결 : 단어 경계를 기준으로 매칭해 , 키워드가 온전한 단어로만 인식되도록 함 (\bcome\b)
- 작은 Vosk 모델은 정확히 "turn to me" 라는 문구를 거의 반환하지 않았고 , 대신 "turn", "turn to", "turned to me" 같은 문구만을 반환
  - 해결 : 네 가지 명령 중 어근이 turn 으로 시작하는 것은 "turn to me" 뿐이므로 , turn 뒤에 임의의 단어 문자가 올 수 있도록 매칭 (\bturn\w*\b)
- Python re 모듈 사용

트러블슈팅 — 4. Stale Bearing (낡은 일회성 방향) 버그

문제 :

- 로봇이 움직인 뒤 명령을 다시 내리면 , 로봇은 약간 틀어진 헤딩을 향해 회전했고 , 이 작은 오차는 반복해서 명령을 내려도 계속 남아 있었다 . 좌표를 다시 입력해야만 그것이 바로잡혔다 .
- 로봇이 음원 지점을 지나쳐 가버린 뒤 명령을 다시 내리면 , 뒤로 돌아서지 않고 계속 앞으로 나아가며 음원에서 멀어지는 일이 있었다 .

원인 분석 :

- 오차가 명령마다 계속 남는 현상과 좌표를 다시 입력하면 해결된다는 사실을 함께 보면 , 발행된 방위가 낡아 있었음을 알 수 있다 .
- 즉 , 목표 방향을 오래된 로봇 위치를 기준으로 좌표 입력 시 한 번만 계산했기 때문

해결 :

- sound_source_node 가 좌표 저장 , /odom 지속 구독 , 로봇의 현재 자세를 기준으로 다시 계산한 방향을 10 Hz 로 재게시하도록 변경

결과 :

- sound_direction 에 실리는 값은 항상 최신 상태다 .
- 명령 시점에 항상 최신 방향을 사용 가능

트러블슈팅 — 5. 로봇 움직임 도중 좌표 이동 시 동작 구현

- 선택지 : 좌표가 바뀔 때마다 목표 갱신 vs 새 명령이 들어올 때만 목표 갱신
  - 로봇이 "come here" 를 수행하고 있는 동안 새로운 좌표를 입력하면 , 로봇이 즉시 새 지점을 향해 이동 ?
- 최종 선택 : 새 명령이 들어올 때만 갱신 (target latching)
  - 조용한 좌표 변경은 로봇을 움직이지 않는다 . 명령이 다시 발화될 때만 변경된 좌표를 목표로 하게 된다 .

평가 및 결과

구현 이후 아래의 네 가지 목표가 모두 수행됨을 확인하였다 .

- (1) 사용자가 입력한 음성에서 명령어를 인식 ,
- (2) 사용자가 입력한 음원의 방향을 계산 ,
- (3) 피드백 루프를 통해 목표 방향까지 안정적으로 회전 ,
- (4) 네 가지 명령이 소리의 방향을 기준으로 실행되도록 동작 구현

평가 및 결과 — 기능 평가

회전 기능 평가

컨트롤러의 회전 정확도는 정량적으로 측정했다 .

🎬 **방법 :**

- 명령 (turn to me) 이후 로봇이 처음 정렬 상태에 도달했을 때마다 명령된 목표 헤딩 , 로봇의 최종 yaw, 그리고 그로부터 계산된 헤딩 오차를 기록
- 20 회 "turn to me" 실험
- 각 시험의 오차 절댓값을 평균내어 계산

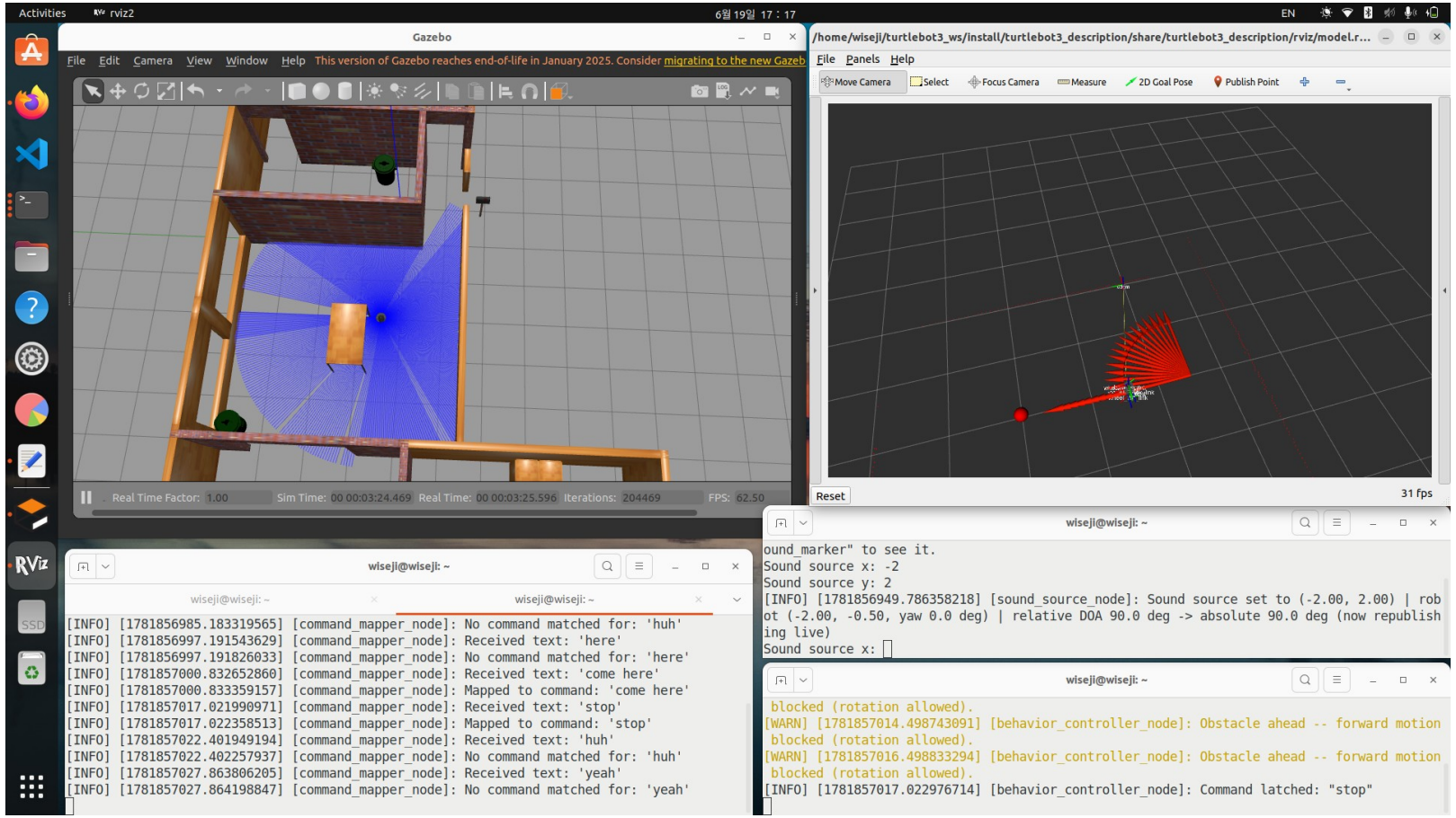
🎬 **결과 :**

- 평균 절대 방향 오차 = 2.59°
- ANGLE_TOL = 0.05 rad ≈ 2.86° 보다 약간 작음
- 측정 결과가 제어 로직과 일관됨을 보여줌

	A	B	C	D	E	F
1	stamp_sec	command	target_deg	final_yaw_deg	error_deg	
2	1781852655.461	turn to me	-36.871	-34.35	-2.521	2.521
3	1781852687.261	turn to me	-9.461	-11.936	2.476	2.476
4	1781852721.661	turn to me	153.444	150.616	2.828	2.828
5	1781852742.861	turn to me	-26.387	-29.158	2.772	2.772
6	1781852757.861	turn to me	-45.022	-42.61	-2.412	2.412
7	1781852779.161	turn to me	-90.032	-87.413	-2.619	2.619
8	1781852799.861	turn to me	51.389	48.825	2.564	2.564
9	1781852856.661	turn to me	14.048	16.48	-2.431	2.431
10	1781852872.061	turn to me	9.462	11.892	-2.429	2.429
11	1781852889.961	turn to me	123.683	121.081	2.602	2.602
12	1781852912.661	turn to me	73.761	76.381	-2.62	2.62
13	1781852929.361	turn to me	52.428	55.004	-2.576	2.576
14	1781852938.861	turn to me	47.723	50.377	-2.655	2.655
15	1781852953.861	turn to me	-121.602	-119.155	-2.447	2.447
16	1781852970.461	turn to me	-153.247	-150.751	-2.496	2.496
17	1781853010.961	turn to me	102.533	105.175	-2.641	2.641
18	1781853061.361	turn to me	23.434	26.1	-2.666	2.666
19	1781853069.861	turn to me	149.418	146.725	2.693	2.693
20	1781853082.761	turn to me	-66.013	-68.798	2.784	2.784
21	1781853094.661	turn to me	-34.288	-36.787	2.499	2.499
22						2.58655

장애물 정지 기능 평가

초기 목표 달성 후 장애물 정지 기능을 추가하였다 .
시뮬레이션 환경의 장애물을 향해 로봇을 주행시키며 정성적으로 평가한 결과 , 해당 기능 또한 안정적으로 동작함을 확인할 수 있었다 .



한계점 및 향후 연구 — 한계점

현재 시스템은 몇 가지 단순화에 의해 범위가 제한된다 .

- 명령어를 네 가지로 한정
- 음원 방향 : 실제 마이크 어레이 대신 좌표 입력 사용
  - 시뮬레이션에 실제 오디오가 없기 때문에 택한 우회 방법이지만 ,
  - 그 결과 실제 환경에서 음원 방향을 추정할 때의 실질적 어려움 (전후 모호성 , 좌우 부호 규약 , 잡음) 은 모델링되지 않으며 , 시스템은 깨끗하고 정확한 방향을 전제로 한다 .
- 거리 기반 자동 정지 미구현
  - stop 명령 또는 장애물이 없으면 계속 전진
- 시뮬레이션 환경에서만 검증
  - 실제 환경에서의 오차 미반영

한계점 및 향후 연구 — 향후 연구

- 자연어 처리를 이용해 다양한 표현을 명령으로 해석할 수 있도록 구현
- 실제 마이크 어레이 기반 DOA (Direction of Arrival) 추정
- 자동 도착 정지 기능
- Nav2 연동을 통한 진정한 navigation 구현
- 물리적 하드웨어에 배포

결론

이 프로젝트는 실내 로봇 제어 시스템을 구축하여 , 사람이 소리를 들을 때처럼 소리의 방향을 기준으로 움직이도록 음성 명령에 반응하게 만드는 것을 목표로 했다 . ROS2 Humble, Gazebo Classic, TurtleBot3 위에 네 개의 협력적인 Python 노드로 구축된 최종 시스템은 그 목표를 네 가지 명령에 대해 달성한다 . 즉 , 오프라인으로 음성을 인식하고 , 인식기 잡음에도 견고하게 명령으로 매핑하며 , 음원 방향을 도출해 지속적으로 유지하고 , 측정된 평균 약 2.6° 오차로 로봇을 목표 방향에 정렬하는 비례 피드백 제어기로 각 동작을 실행한다 . LiDAR 기반 안전 계층은 로봇이 장애물에 충돌하지 않도록 막으면서도 회전 능력은 보존한다 .